Probabilistic Latent Reasoning via Diffusion

1 Motivation

Recent recursive latent reasoning models, such as HRM, TRM, PTRM, EqR, and Attractor Models, suggest that strong reasoning can emerge from repeatedly refining a compact latent workspace rather than emitting long textual chain-of-thought traces. A common abstraction is:

```
tokens / grid / symbols
   -> encoder
latent workspace z^{(0)}
   -> repeated update: z^{(r+1)} = F_theta(z^{(r)}, x, r)
   -> decoder / projection
answer tokens / grid / action
```

This proposal asks whether the deterministic or weakly stochastic update in these models can be replaced by a learned probabilistic latent diffusion process. The goal is not to generate continuous text rationales, as in latent CoT diffusion or flow models. Instead, the goal is to model the distribution over latent solution workspaces:

$$p_\theta(z_{\text{solution}} \mid x),$$

where samples from this distribution decode to valid answers.

Central hypothesis. Reasoning can be represented as stochastic refinement of a latent solution workspace. Diffusion-style dynamics may provide a principled way to explore multiple reasoning basins, refine partial solutions, and select high-quality final states.

2 Core Framework

Given an input problem x, such as a token sequence, symbolic grid, Sudoku board, maze, or ARC task, an encoder maps the input into a conditioning representation:

$$c = E_\phi(x).$$

The model maintains a latent workspace $z^{(r)}$, where r indexes recursive reasoning/refinement steps. This workspace should be expressive enough to store partial hypotheses about the solution, but compact enough that repeated refinement is efficient.

The high-level computation is:

```
x
  -> c = E_phi(x)
  -> z^{(0)} ~ initial proposal/noise
  -> z^{(1)}, ..., z^{(R)} by learned stochastic refinement
  -> y_hat = D_psi(z^{(R)}, c)
```

Here R is the reasoning/refinement horizon. We use the recursive-reasoning convention: $z^{(0)}$ is the initial noisy proposal, and $z^{(R)}$ is the final denoised/refined latent solution state. This avoids the usual diffusion convention, where z_0 is clean and z_T is noisy.

The learned stochastic update is

$$z^{(r+1)} = F_\theta(z^{(r)}, c, r) + \sigma_r \epsilon_r, \qquad \epsilon_r \sim \mathcal{N}(0, I).$$

Equivalently,

$$z^{(r+1)} \sim p_\theta(z^{(r+1)} \mid z^{(r)}, c, r).$$

After refinement, a decoder projects the final latent workspace to the answer:

$$\hat{y} = D_\psi(z^{(R)}, c).$$

3 Relationship to Existing Models

TRM. TRM uses deterministic recursive refinement:

$$z_{t+1} = F_\theta(z_t, x).$$

It learns an iterative correction operator. Test-time scaling means running more recursive steps.

PTRM. PTRM keeps the trained TRM fixed but injects Gaussian noise during inference:

$$z_{t+1} = F_\theta(z_t + \epsilon_t, x).$$

It samples multiple rollouts and selects the best one using a Q head. Stochasticity is an inference-time exploration trick.

Attractor Models. Attractor Models replace finite unrolling with a fixed-point solve:

$$z^\star = F_\theta(z^\star, x).$$

They ask for the equilibrium of the latent update rather than a fixed number of iterations.

Proposed Latent Diffusion Reasoner. The proposed model learns the stochastic transition itself:

$$z^{(r+1)} \sim p_\theta(z^{(r+1)} \mid z^{(r)}, x, r).$$

Thus, stochastic exploration is not bolted onto a deterministic model. It is part of the learned reasoning dynamics.

4 Modeling Choices

4.1 Latent Workspace

The latent workspace can be task-specific or generic. For structured tasks, natural choices include:

- Sudoku: latent tensor over cells and symbols.
- Maze: latent grid over path, value, and connectivity features.

- ARC: latent grid with object, color, transformation, and correspondence features.

For language or symbolic tasks, the workspace could be a sequence of latent thought/workspace vectors. A simple first version should use a structured latent tensor

$$z^{(r)} \in \mathbb{R}^{H \times W \times d}$$

for grid tasks. This makes the decoder and losses easy to define.

4.2 Conditional Denoising Transition

The recursive updates are treated as learned denoising/refinement steps in latent workspace space. A diffusion-style transition can be parameterized by a Transformer, U-Net, or grid-aware attention model:

$$F_\theta(z^{(r)}, c, r).$$

For a stochastic denoising update,

$$z^{(r+1)} = F_\theta(z^{(r)}, c, r) + \sigma_r \eta, \qquad \eta \sim \mathcal{N}(0, I).$$

Thus $z^{(0)}$ can be interpreted as a noisy proposal and $z^{(R)}$ as the denoised solution latent.

During training, however, it is useful to introduce a separate diffusion-time variable $\tau \in [0, 1]$. This variable indexes interpolation between a clean latent solution target $z_\star$ and Gaussian noise ϵ; it should not be confused with the recursive step r.

Alternatively, use flow matching:

$$\frac{dz(\tau)}{d\tau} = v_\theta(z(\tau), c, \tau),$$

and sample with an ODE/SDE solver. Flow matching may be cleaner because it frames reasoning as continuous latent transport from noise to solution.

4.3 Decoder

The decoder maps the final latent workspace to the output:

$$\hat{y} = D_\psi(z^{(R)}, c).$$

For Sudoku,

$$D_\psi(z^{(R)}) \in \mathbb{R}^{9 \times 9 \times 9}.$$

For maze,

$$D_\psi(z^{(R)}) \in \mathbb{R}^{H \times W \times \text{action/path}}.$$

For ARC,

$$D_\psi(z^{(R)}) \in \mathbb{R}^{H_{\text{out}} \times W_{\text{out}} \times \text{colors}}.$$

The decoder can also be applied to intermediate states to encourage monotonic refinement:

$$\hat{y}^{(r)} = D_\psi(z^{(r)}, c).$$

5 Architecture Details

For Sudoku, the main implementation should use a Transformer over 81 cell tokens. This aligns the model with TRM-style sequence processing while allowing us to reuse the same token structure for the VAE, latent flow model, decoder, and Q head.

5.1 Tokenization and Embeddings

Represent the puzzle as 81 cell tokens. Each input cell has value

$$x_i \in \{0, 1, \ldots, 9\},$$

where 0 denotes an empty cell. The completed solution has

$$y_i^\star \in \{1, \ldots, 9\}.$$

For the main comparable baseline, use value embeddings and 1D RoPE over the flattened 81-cell sequence, matching the TRM setting:

$$e_i^x = E_x(x_i), \qquad i = 1, \ldots, 81.$$

RoPE is applied inside each self-attention layer to the query and key vectors:

$$Q_i, K_i \leftarrow \mathrm{RoPE}(Q_i, K_i; i).$$

To stay close to TRM, the baseline should not use learned absolute position embeddings, 2D row/column position embeddings, or Sudoku-specific row/column/box attention bias. Full non-causal self-attention over the 81 flattened cells is sufficient. Learned absolute positions, 2D RoPE, and row/column/box relation attention bias can be added later as ablations, but they should not be part of the headline comparison.

5.2 Conditional VAE Architecture

The VAE should use a structured per-cell latent rather than a single global vector:

$$z_\star \in \mathbb{R}^{81 \times d_z}.$$

The VAE encoder receives both the puzzle and the completed solution:

$$e_i^{\mathrm{enc}} = E_x(x_i) + E_y(y_i^\star).$$

A small Transformer encoder produces hidden states

$$h^{\mathrm{enc}} = T_\omega^{\mathrm{enc}}(e^{\mathrm{enc}}),$$

and per-cell Gaussian parameters

$$\mu_i, \log \sigma_i^2 = W_{\mu,\sigma} h_i^{\mathrm{enc}}.$$

The latent target is sampled by reparameterization:

$$z_{\star,i} = \mu_i + \sigma_i \epsilon_i, \qquad \epsilon_i \sim \mathcal{N}(0, I).$$

The VAE decoder conditions on the puzzle and the latent state:

$$e_i^{\mathrm{dec}} = W_z z_{\star,i} + E_x(x_i).$$

A Transformer decoder/backbone produces per-cell logits:

$$h^{\mathrm{dec}} = T_\psi^{\mathrm{dec}}(e^{\mathrm{dec}}), \qquad \ell_i = W_o h_i^{\mathrm{dec}} \in \mathbb{R}^9.$$

A reasonable first configuration is:

- VAE encoder: 4 Transformer layers, $d_{\mathrm{model}} = 128$, 4 attention heads.
- VAE decoder: 4 Transformer layers, $d_{\mathrm{model}} = 128$, 4 attention heads.
- Latent dimension: $d_z = 64$.
- Positional encoding: 1D RoPE over flattened cell positions $1, \ldots, 81$.

5.3 Latent Normalization

Following the spirit of latent diffusion models, the latent space should be normalized before flow training. After VAE pretraining, estimate the empirical mean and standard deviation of encoded latents:

$$\mu_z = \mathbb{E}[z_\star], \qquad \sigma_z = \mathrm{Std}(z_\star).$$

Use normalized latents for flow matching:

$$\tilde{z}_\star = \frac{z_\star - \mu_z}{\sigma_z}.$$

At decoding time, map a generated normalized latent back to the VAE decoder scale:

$$z = \sigma_z \tilde{z} + \mu_z.$$

This makes the Gaussian noise prior and flow-matching interpolation better matched to the learned latent distribution.

5.4 Latent Flow Reasoner Architecture

The latent flow reasoner predicts a velocity field over the 81 latent tokens:

$$v_\theta(z_\tau, c, \tau) \in \mathbb{R}^{81 \times d_z}.$$

Its input token at cell i is

$$a_i = W_z z_{\tau,i} + E_x(x_i) + E_\tau(\tau),$$

where $E_\tau(\tau)$ is a sinusoidal or MLP time embedding broadcast to all cell tokens. A Transformer backbone computes

$$h^\theta = T_\theta(a),$$

and the velocity prediction is

$$v_{\theta,i} = W_v h_i^\theta.$$

The same Transformer weights are reused at every inference/refinement step. A reasonable first configuration is:

- Reasoner: 6 Transformer layers, $d_{\text{model}} = 256$, 8 attention heads.
- Latent dimension: $d_z = 64$.
- Conditioning: add puzzle embeddings to every cell token.
- Time embedding: sinusoidal embedding followed by a small MLP.
- Positional encoding: 1D RoPE over flattened cell positions $1, \ldots, 81$.
- Attention: full non-causal self-attention, no Sudoku-specific relation bias in the main baseline.

For Euler inference from noise to clean latent, discretize τ from 1 to 0. If the learned vector field is trained with target $\epsilon - z_\star$, the update direction should move opposite the increasing-τ field:

$$z_{\tau - \Delta\tau} = z_\tau - \Delta\tau \, v_\theta(z_\tau, c, \tau).$$

The sign convention should be verified in implementation by checking whether decoded cell accuracy improves as τ decreases.

5.5 Q-Head Architecture

The Q head should be lightweight. The simplest version pools the reasoner hidden states and predicts a scalar:

$$q = \mathrm{MLP}\left(\frac{1}{81}\sum_{i=1}^{81} h_i^\theta\right).$$

An alternative is to prepend a learned $[Q]$ token to the reasoner sequence and use its final hidden state:

$$q = \mathrm{MLP}(h_Q^\theta).$$

The $[Q]$-token version may aggregate global consistency more naturally, but mean pooling is the cleanest first baseline. The Q head should be trained jointly with the reasoner and used primarily for $K > 1$ sample selection; early stopping remains an ablation.

6 Training Objectives

The main difficulty is that we observe input-output pairs $(x, y^\star)$, but not the true latent reasoning trajectory. We therefore use a two-stage training pipeline. First, learn a stable latent solution space with a conditional VAE or autoencoder. Second, freeze this latent space and train the latent reasoner and Q head jointly, following the spirit of TRM's joint answer-and-halt training.

6.1 Stage 1: Conditional VAE for Solution States

The conditional VAE defines the clean latent target used by the flow model. Given a puzzle x and its completed solution $y^\star$, the encoder produces a distribution

$$q_\omega(z_\star \mid x, y^\star) = \mathcal{N}(\mu_\omega(x, y^\star), \mathrm{diag}(\sigma_\omega^2(x, y^\star))).$$

We sample

$$z_\star = \mu_\omega(x, y^\star) + \sigma_\omega(x, y^\star) \odot \xi, \qquad \xi \sim \mathcal{N}(0, I),$$

and decode it back to a solution grid:

$$\ell_\star = D_\psi(z_\star, E_\phi(x)).$$

The VAE is trained with reconstruction plus KL regularization:

$$\mathcal{L}_{\mathrm{VAE}} = \mathrm{CE}(\ell_\star, y^\star) + \beta\,\mathrm{KL}\left(q_\omega(z_\star \mid x, y^\star) \,\|\, \mathcal{N}(0, I)\right).$$

For the main experiment, the VAE encoder and decoder are frozen after this stage. This keeps the target latent distribution $z_\star$ fixed while the reasoner learns the transport/refinement dynamics. A later ablation can unfreeze the decoder, or the last decoder layers, with a much smaller learning rate; the VAE encoder should remain frozen unless an explicit reconstruction/KL anchor is kept.

6.2 Stage 2: Joint Reasoner and Q-Head Training

The reasoner is trained on top of the frozen VAE latent space. For each training pair $(x, y^\star)$, obtain a clean latent target

$$z_\star \sim q_\omega(z \mid x, y^\star), \qquad c = E_\phi(x).$$

Then train the latent flow/reasoner to transport noisy proposals toward $z_\star$, while also decoding its final and intermediate states to the correct board. Following LaDiR, reasoner training should include both teacher-forced flow matching and rollout training, because pure teacher forcing creates a mismatch between oracle interpolation states seen during training and self-generated states seen during inference.

Final answer loss. The final decoded state should solve the task:

$$\ell^{(R)} = D_\psi(z^{(R)}, c), \qquad \mathcal{L}_{\text{answer}} = \text{CE}(\ell^{(R)}, y^\star).$$

Flow matching loss. Following the LaDiR-style construction, sample Gaussian noise

$$\epsilon \sim \mathcal{N}(0, I),$$

and interpolate linearly between the clean latent solution $z_\star$ and noise:

$$z_\tau = (1-\tau)z_\star + \tau\epsilon, \qquad \tau \sim \mathcal{U}(0,1).$$

The target velocity is

$$u^\star(z_\tau, \tau) = \frac{dz_\tau}{d\tau} = \epsilon - z_\star.$$

Train a conditional vector field to predict this velocity:

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{\tau,\epsilon}\left[\|v_\theta(z_\tau, c, \tau) - (\epsilon - z_\star)\|^2\right].$$

At inference time, integrate the learned vector field from noise toward the clean solution direction. If $\tau = 1$ denotes noise and $\tau = 0$ denotes clean data, this can be written as an ODE solve from $\tau = 1$ to $\tau = 0$:

$$\frac{dz(\tau)}{d\tau} = v_\theta(z(\tau), c, \tau), \qquad z(1) \sim \mathcal{N}(0, I).$$

Equivalently, after discretization and reindexing by recursive step r, this gives the update sequence

$$z^{(0)} \to z^{(1)} \to \cdots \to z^{(R)},$$

where $z^{(0)}$ is noisy and $z^{(R)}$ is clean/refined.

Noise-prediction version. One could also use a DDPM-style objective:

$$z_\tau = \alpha_\tau z_\star + \sigma_\tau \epsilon, \qquad \mathcal{L}_{\text{diff}} = \|\epsilon - \epsilon_\theta(z_\tau, c, \tau)\|^2.$$

However, the flow-matching version is often a cleaner match to our intended interpretation: reasoning is continuous transport from an initial noisy proposal to a structured solution latent.

6.3 Rollout Training

Pointwise flow matching trains the vector field on oracle interpolation states $z_\tau = (1-\tau)z_\star + \tau\epsilon$. At inference, however, the model starts from pure noise and repeatedly applies its own predicted velocity. Small vector-field errors can move the latent state off the oracle interpolation path, causing error accumulation. LaDiR addresses the analogous mismatch with a second rollout-training stage; we should use the same idea for Sudoku.

In rollout training, sample

$$z^{(0)} \sim \mathcal{N}(0, I),$$

and unroll the same solver used at inference:

$$z^{(r+1)} = z^{(r)} - \Delta\tau \, v_\theta(z^{(r)}, c, \tau_r), \qquad \tau_r = 1 - \frac{r}{R_{\text{train}}}, \qquad \Delta\tau = \frac{1}{R_{\text{train}}}.$$

After R_{train} steps, decode the generated latent:

$$\ell_{\text{roll}} = D_\psi(z^{(R_{\text{train}})}, c),$$

and apply the Sudoku answer loss:

$$\mathcal{L}_{\text{rollout}} = \text{CE}(\ell_{\text{roll}}, y^\star).$$

Gradients should pass through the unrolled denoising trajectory. This directly trains the reasoner on the distribution it will encounter at test time.

The rollout stage should keep the flow-matching loss active:

$$\mathcal{L}_{\text{FM}} + \lambda_{\text{rollout}} \mathcal{L}_{\text{rollout}},$$

because the flow loss anchors the model to the VAE latent geometry and reduces the risk of latent collapse or arbitrary decoder-hacking trajectories.

Practical schedule. Start with teacher-forced flow matching, then enable rollout training:

1. Warm up with pointwise $\mathcal{L}_{\text{FM}}$, answer loss from one-step denoising, and Q-head supervision.
2. Enable rollout training with small R_{train}, such as 2 or 4.
3. Increase R_{train} only after rollout cell accuracy improves.
4. Evaluate with larger inference budgets $R \in \{8, 16, 32\}$.

Use a small initial rollout weight, for example $\lambda_{\text{rollout}} = 0.1$, and monitor both flow loss and rollout CE.

6.4 Intermediate Decoding Loss (Optional)

To make the diffusion trajectory behave like reasoning rather than arbitrary denoising, apply supervision to intermediate predictions:

$$\mathcal{L}_{\text{intermediate}} = \sum_{r=1}^{R} w_r \, \text{CE}(D_\psi(z^{(r)}, c), y^\star).$$

Weights w_r can emphasize later, cleaner states.

6.5 Joint Q-Head Loss

The Q head is trained jointly with the reasoner, not as a separate post-hoc verifier. It estimates whether a latent state decodes to an exactly correct answer:

$$q^{(r)} = Q_\eta(z^{(r)}, c).$$

Following the TRM-style target, define

$$h^{(r)} = \mathbf{1}\left\{\arg\max D_\psi(z^{(r)}, c) = y^\star\right\}.$$

Then train the Q head with

$$\mathcal{L}_Q = \mathrm{BCE}(q^{(r)}, h^{(r)}).$$

For tasks with partial correctness metrics, $h^{(r)}$ can be replaced or supplemented by a soft reward:

$$\rho^{(r)} = \mathrm{score}(D_\psi(z^{(r)}, c), y^\star),$$

and Q_η can be trained by regression in an ablation. The default Sudoku setting should use exact decoded correctness, matching the TRM convention.

6.6 Utility of the Q Head

The Q head is useful but not structurally required for the latent diffusion reasoner. The core reasoner can always be evaluated with a fixed number of refinement steps R and a single sample $K = 1$. We keep the Q head because it supports adaptive and multi-sample inference without changing the decoder or evaluation metric.

Primary role: sample selection. When $K > 1$, each stochastic trajectory produces a candidate board. The Q head ranks these candidates by predicted exact correctness:

$$k^\star = \arg\max_k Q_\eta(z^{(R,k)}, c).$$

This is the cleanest use of Q in our setting, analogous to PTRM's use of a learned correctness signal to select among stochastic latent rollouts.

Optional role: inference-time early stopping. The Q head can also support adaptive computation at inference time. After a minimum number of refinement steps $R_{\min}$, stop if

$$\sigma(Q_\eta(z^{(r)}, c)) > \gamma.$$

This trades accuracy for compute and should be reported as an ablation against fixed-R inference.

Training-time early stopping. We should not use Q-based early stopping in the first training baseline. Early in training, the Q head is poorly calibrated and most intermediate states are incorrect, so halting can introduce a biased or unstable training distribution. A safer schedule is:

1. Train with fixed R, while supervising the Q head at all observed steps.
2. After Q calibration improves, optionally allow training-time early stopping with exploration or random continuation.
3. Compare fixed-R training against Q-halting training as an efficiency ablation.

6.7 Total Stage-2 Objective

The reasoner and Q head are trained jointly with

$$\mathcal{L} = \mathcal{L}_{\text{answer}} + \lambda_{\text{FM}}\mathcal{L}_{\text{FM}} + \lambda_{\text{rollout}}\mathcal{L}_{\text{rollout}} + \lambda_{\text{int}}\mathcal{L}_{\text{intermediate}} + \lambda_Q\mathcal{L}_Q.$$

7 Inference

At test time, sample multiple latent trajectories:

$$z^{(0,k)} \sim \mathcal{N}(0, I), \qquad k = 1, \ldots, K.$$

Run the learned refinement dynamics:

$$z^{(0,k)} \to z^{(1,k)} \to \cdots \to z^{(R,k)}.$$

Decode each candidate:

$$\hat{y}^{(k)} = D_\psi(z^{(R,k)}, c).$$

Select the final answer by highest jointly trained Q score, majority vote, lowest constraint violation, or lowest diffusion/energy residual. The simplest selector is

$$k^\star = \arg\max_k Q_\eta(z^{(R,k)}, c), \qquad \hat{y} = \hat{y}^{(k^\star)}.$$

This gives two test-time scaling axes:

- Depth: increase diffusion/refinement steps R.
- Width: increase sampled trajectories K.

8 First Experimental Plan

8.1 Start with Sudoku

Sudoku is the cleanest first benchmark because inputs and outputs have the same grid structure, correctness is exactly checkable, constraint violations are easy to measure, TRM/HRM/PTRM already report strong results, and the latent workspace can be a $9 \times 9 \times d$ tensor.

Initial architecture.

- Conditional VAE: 4-layer Transformer encoder and 4-layer Transformer decoder over 81 cell tokens.
- Latent workspace: per-cell latent $z \in \mathbb{R}^{81\times 64}$, normalized before flow training.
- Flow reasoner: 6-layer Transformer vector field, $d_{\text{model}} = 256$, 8 attention heads.
- Decoder: frozen VAE decoder followed by a linear head to 9 symbols per cell.
- Q head: mean-pooled reasoner hidden states to scalar correctness score; $[Q]$-token variant as an ablation.
- Positional encoding: 1D RoPE over flattened cell positions, matching TRM.
- Attention: full non-causal self-attention without Sudoku-specific relation bias in the main baseline.

Training setup.

- Stage 1: train a conditional VAE on puzzle-solution pairs to produce clean latent solution states $z_\star$.
- Freeze the VAE encoder and decoder for the main reasoner experiment.
- Stage 2a: warm up the latent reasoner and Q head with teacher-forced flow matching, one-step answer decoding, and TRM-style exact-correctness Q targets.
- Stage 2b: enable LaDiR-style rollout training from pure noise, backpropagating Sudoku CE through the unrolled denoising trajectory.
- Start rollout training with $R_{\text{train}} \in \{2, 4\}$ and $\lambda_{\text{rollout}} \approx 0.1$, then increase R_{train} after rollout cell accuracy improves.
- Use the Q head for $K > 1$ sample selection in the main multi-sample setting.
- Treat Q-based early stopping as an inference-time ablation first, and training-time early stopping only after fixed-R training is stable.
- Add intermediate decoding after the basic model works.
- Ablate small-learning-rate decoder finetuning after the frozen-VAE baseline is stable.

Baselines.

- Direct Transformer.
- TRM.
- PTRM.
- Attractor-style fixed-point TRM.
- Latent Diffusion Reasoner, $K = 1$.
- Latent Diffusion Reasoner, $K > 1$, Q-head selection.
- Latent Diffusion Reasoner, Q-head early stopping.
- Latent Diffusion Reasoner without Q head.

Metrics.

- Exact puzzle accuracy.
- Cell accuracy.
- Number of Sudoku constraint violations.
- Accuracy versus refinement depth R.
- Accuracy versus width K.
- Calibration of Q score.
- Compute saved by Q-head early stopping.

8.2 Sudoku Implementation Details

For the first Sudoku implementation, we should follow the HRM/TRM/Attractor evaluation convention as closely as possible. The model should be evaluated as a direct neural grid predictor, not as a neural proposal followed by a symbolic Sudoku solver.

Input and output representation. The input puzzle is represented as a length-81 sequence or 9×9 grid:

$$x_i \in \{0, 1, \ldots, 9\},$$

where 0 denotes an empty cell and $1, \ldots, 9$ denote givens. The target is the completed solution grid

$$y_i^\star \in \{1, \ldots, 9\}, \qquad i = 1, \ldots, 81.$$

The encoder produces a conditioning representation

$$c = E_\phi(x),$$

and the latent diffusion reasoner refines a latent workspace

$$z^{(0)} \to z^{(1)} \to \cdots \to z^{(R)}.$$

Decoder and logits. The decoder maps the final refined latent state to cell-wise logits:

$$\ell = D_\psi(z^{(R)}, c), \qquad \ell \in \mathbb{R}^{81 \times 9}.$$

The predicted board is obtained by independent argmax over the 9 digits in each cell:

$$\hat{y}_i = 1 + \arg \max_{d \in \{1,\ldots,9\}} \ell_{i,d}.$$

No Sudoku solver, exact-cover projection, ILP layer, or constraint-programming postprocessor is used for the main result.

Training loss. The final answer loss is ordinary cell-wise cross-entropy against the completed solution:

$$\mathcal{L}_{\text{answer}} = \frac{1}{81} \sum_{i=1}^{81} \text{CE}(\ell_i, y_i^\star).$$

If intermediate decoding is used, the same decoder can be applied to intermediate states:

$$\ell^{(r)} = D_\psi(z^{(r)}, c),$$

with

$$\mathcal{L}_{\text{intermediate}} = \sum_{r=1}^{R} w_r \frac{1}{81} \sum_{i=1}^{81} \text{CE}(\ell_i^{(r)}, y_i^\star).$$

This supervises each intermediate decoded board against the final solution, but it does not require or impose human-like filling trajectories.

Exact accuracy. The primary Sudoku metric is exact puzzle accuracy:

$$\text{ExactAcc} = \frac{1}{N} \sum_{n=1}^{N} \mathbf{1}\left\{ \hat{y}_i^{(n)} = y_i^{\star(n)} \text{ for all } i = 1, \ldots, 81 \right\}.$$

A puzzle is counted as correct only if all 81 predicted cells match the ground-truth solution. Cell accuracy can be reported as a secondary metric:

$$\text{CellAcc} = \frac{1}{81N} \sum_{n=1}^{N} \sum_{i=1}^{81} \mathbf{1}\{\hat{y}_i^{(n)} = y_i^{\star(n)}\}.$$

Constraint diagnostics. Sudoku constraints should be measured, but not used to define the headline accuracy. For a decoded board $\hat{y}$, define row, column, and box violation counts as the number of missing or repeated digits in the 27 Sudoku units. A solved board has zero violations. These diagnostics answer a different question from exact accuracy: whether the model has learned Sudoku structure even when it does not exactly match the target solution.

Optionally, we can add a soft constraint regularizer during training:

$$\mathcal{L}_{\text{sudoku}} = \mathcal{L}_{\text{row}} + \mathcal{L}_{\text{col}} + \mathcal{L}_{\text{box}},$$

where, for probabilities $p_{i,d} = \text{softmax}(\ell_i)_d$,

$$\mathcal{L}_{\text{row}} = \sum_{\text{row } u} \sum_{d=1}^{9} \left(\sum_{i \in u} p_{i,d} - 1 \right)^2,$$

and similarly for columns and boxes. This regularizer is optional and should be ablated, because the cleanest comparison to HRM/TRM/Attractor uses only supervised prediction losses and exact-match evaluation.

Multi-sample inference. For $K > 1$, each trajectory produces logits $\ell^{(k)}$, an argmax board $\hat{y}^{(k)}$, and a jointly trained Q score $Q_\eta(z^{(R,k)}, c)$. The main selection rule can be

$$k^\star = \arg\max_k Q_\eta(z^{(R,k)}, c), \qquad \hat{y} = \hat{y}^{(k^\star)}.$$

Constraint violation counts may be used as an auxiliary selection signal in a separate ablation, but the default version should use the jointly trained Q head so that the method remains comparable to PTRM-style latent rollout selection.

8.3 Then Maze

Maze tests whether the model learns global path connectivity rather than local symbol constraints. Useful outputs include a path mask, next-action policy, and distance-to-goal or value field.

8.4 Then ARC-Style Grids

ARC is the most interesting but should come later because the latent target and decoder are harder. A first ARC experiment should use fixed-size preprocessed grids and avoid variable-size output complications.

9 Key Research Questions

1. Does learned stochastic refinement outperform deterministic recursion at the same parameter count?
2. Is width scaling with K more efficient than depth scaling with R?
3. Does the learned Q head reliably identify correct latent solution basins?
4. Does diffusion-style training produce smoother or more task-aligned latent landscapes than TRM/PTRM?
5. Can the model solve harder instances by sampling more trajectories without retraining?
6. Does the intermediate trajectory become interpretable as progressive reasoning, or is it only a generative denoising path?

10 Main Risks

The largest risk is that diffusion learns to generate answer-like latents without genuinely improving reasoning. If the final decoder is too strong, the diffusion trajectory may become decorative.

Mitigations.

- Use small decoders.
- Decode intermediate states.
- Measure constraint violations over time.
- Compare against deterministic recurrence with the same architecture.
- Use jointly trained Q-head and residual diagnostics.

A second risk is latent-space drift. If the VAE encoder or decoder changes while the reasoner is learning, then the clean target distribution $z_\star$ moves, making the flow-matching problem unstable.

Mitigations.

- Freeze the VAE encoder and decoder in the main experiment.
- Finetune only the decoder, or final decoder layers, with a much smaller learning rate as an ablation.
- Keep a reconstruction/KL anchor if any VAE component is unfrozen.